SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: TravelSphere

Submitted By: Tanisha Rathod

Savan Patel
Vivek Hadiya

Institute: RK University

Date: 04/05/2026

1. Introduction

1.1 Purpose

The primary purpose of this Software Requirements Specification (SRS) document is to meticulously define all functional and non-functional requirements for "TravelSphere," a comprehensive web-based travel booking system. TravelSphere is designed as a full-stack application to serve as a centralized platform facilitating travel package search, selection, and booking for end-users, while providing robust administrative tools for system management. This document aims to provide all stakeholders—including developers, testers, and project managers—with a clear, unambiguous, and exhaustive understanding of the system's goals and behavior. The system is being built to streamline the process of discovering and booking travel, consolidating information from various providers, and offering a seamless user experience, thereby replacing fragmented and inefficient manual or multi-platform booking processes.

1.2 Document Conventions

All requirements and technical specifications within this document adhere strictly to the conventions of the IEEE standard for SRS. **Requirement Numbering** follows a prefix system: all functional requirements are uniquely identified using the format "REQ-N," where N is a sequentially increasing integer (e.g., REQ-1, REQ-2, REQ-3, etc.). All cross-references and internal terminology maintain consistency. Headings and subheadings are structured hierarchically for ease of navigation. Key terms such as "Traveller," "Admin," and "Package" are capitalized throughout the document for

clarity. **Priority Levels** for features and requirements are denoted as High (Critical for system operation), Medium (Important for usability), or Low (Desirable enhancements).

1.3 Intended Audience and Reading Suggestions

This SRS is intended for multiple distinct audiences. **Software Developers** should focus primarily on Section 3, "System Features," and Section 2.5, "Design and Implementation Constraints," as these sections contain the technical specifications and functional requirements necessary for construction. **System Testers and Quality Assurance personnel** should utilize the entire document, especially Section 3.2, "Stimulus/Response Sequences," to develop comprehensive test plans and acceptance criteria. **Project Managers and Business Analysts** should review Sections 1 and 2 to ensure alignment between the technical implementation and the overarching business objectives and scope. End-users are not the direct audience, but their needs are defined in Section 2.3, "User Classes and Characteristics."

1.4 Project Scope

The scope of the TravelSphere system encompasses a full-stack web application designed for both public access and administrative control. The system will allow unregistered and registered **Travellers** to browse available travel packages (flights, accommodation, tours), register for an account, log in securely, book selected packages through a multi-step process, and submit reviews for completed trips. The system will integrate secure payment gateways (though the detailed specifications of the gateway integration are outside this immediate scope). For **Administrators (Admins)**, the scope includes comprehensive management functionalities: creation, editing, and deletion of travel packages; oversight of all bookings; and moderation/management of user-submitted reviews. The system's core benefit is its ability to offer a unified, efficient, and user-friendly platform for all travel booking needs, reducing reliance on multiple third-party sites.

1.5 References

The following standards and documentation sources were utilized in the creation of this SRS:

- IEEE Standard for Software Requirements Specifications (IEEE Std 830-1998) – Utilized for structural format and content guidelines.
- TravelSphere Project Charter – Outlining initial business goals and high-level scope.
- FileSecurity Standards and Compliance Documentation – Relevant for defining non-functional requirements related to data protection and authentication protocols.

2. Overall Description

2.1 Product Perspective

TravelSphere is a standalone, web-based product intended to function independently, but it is built upon a standard three-tier architecture model. **Tier 1 (Presentation Tier - Frontend):** This is the user interface, developed using React.js, responsible for displaying information and collecting user input. It interacts with the Application Tier via RESTful APIs. **Tier 2 (Application Tier - Backend):** Built on Node.js using the Express framework, this tier contains the core business logic, handling user authentication, request validation, package calculation, and all data processing. It serves as the intermediary between the Frontend and the Database Tier. **Tier 3 (Data Tier - Database):** This tier is a PostgreSQL relational database management system, responsible for persistent storage of all critical system data, including user profiles, travel package details, booking records, and reviews. The system is not a component of a larger system but may interface with external systems like payment gateways and travel APIs in the future.

2.2 Product Features

TravelSphere offers a robust set of features categorized as follows:

- **User Authentication and Profile Management:** Secure registration, login, session management, and the ability for users to view and update their personal information and booking history.
- **Package Browsing and Search:** Advanced search capabilities, filtering options (e.g., by destination, price, duration, date), and detailed package display pages with rich media content.
- **Booking and Payment Processing:** A secure, multi-step workflow for selecting package options, confirming availability, providing passenger details, and facilitating payment submission.
- **Review and Rating System:** A mechanism for verified Travellers to submit textual reviews and numerical ratings for packages they have completed, ensuring transparent feedback.
- **Admin Management Dashboard:** A restricted access interface for administrators to perform CRUD (Create, Read, Update, Delete) operations on packages, manage the status of bookings, and moderate user-generated content.

2.3 User Classes and Characteristics

Traveller

This user class represents the primary end-user of the system. **Characteristics:** Travellers are expected to have basic internet literacy and proficiency in navigating web applications. They are primarily interested in searching, comparing, and booking travel packages quickly and securely. They require a user-friendly interface, reliable performance, and access to accurate information. **Privileges:** Account registration, secure login, profile management, package browsing/searching, booking initiation, payment submission, and posting reviews.

Admin

This user class represents the system maintainers and content managers. Characteristics: Admins are expected to have a higher level of technical understanding and are responsible for data integrity and system operation. They require a dedicated, secure interface with powerful tools for content management and oversight. **Privileges:** Secure Admin login, creation/modification/deletion of travel packages, viewing all user and booking records, updating booking statuses, and moderating/deactivating inappropriate reviews.

2.4 Operating Environment

TravelSphere is designed as a standard web application. **Client-side Environment:** The system must be fully operational and responsive across all major modern web browsers, including Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge (latest two stable versions). It must function on various operating systems (Windows, macOS, Android, iOS). Server-side Environment: The backend must be hosted on a cloud-based or dedicated server environment running a stable Linux distribution (e.g., Ubuntu LTS) with appropriate network security and scaling capabilities. The server must support Node.js (version 18 or later) and PostgreSQL (version 15 or later).

2.5 Design and Implementation Constraints

The following constraints govern the design and implementation of TravelSphere: **Technology Stack Constraint:** The system must be developed using the defined stack: Frontend (React.js), Backend (Node.js/Express), and Database (PostgreSQL). No alternative frameworks or databases are permitted for core development. **Performance Constraint:** All primary pages (e.g., search results, package details) must load within three seconds under a concurrent load of 500 users. **Security Constraint:** The system must enforce HTTPS/SSL across all communication channels. Password storage must utilize a strong, one-way cryptographic hashing algorithm (e.g., bcrypt). **Database Design Constraint:** The database schema must be normalized to at least Third Normal Form (3NF) to ensure data integrity and reduce redundancy.

2.6 User Documentation

The system must be accompanied by comprehensive user documentation. This includes an online, searchable **Help System** accessible from every page, providing context-sensitive guidance on navigating the interface, performing searches, and completing the booking process. A downloadable Traveller Manual will provide step-by-step instructions for account management and booking workflows. A separate Admin Guide will detail the use of the Admin Management dashboard, covering package creation, booking management, and review moderation processes.

2.7 Assumptions and Dependencies

Assumptions:

- **Internet Connectivity:** All users (Travellers and Admins) are assumed to have a stable and reliable internet connection to access the web application.
- **Client Device:** Users are assumed to access the system via a device with a modern, up-to-date web browser.
- **Legal Compliance:** It is assumed that all necessary legal and regulatory approvals for processing online bookings and payments will be obtained prior to deployment.

Dependencies:

- **Payment Gateway API:** The booking feature is critically dependent on the successful integration and continued availability of a third-party payment gateway API for processing transactions.
- **Server and Database Infrastructure:** Continued operation relies on the stable hosting environment and the reliable, low-latency performance of the PostgreSQL database server.

3. System Features

3.1 User Authentication

3.1.1 Description and Priority

The User Authentication feature manages the security and identity of the system's users (Travellers and Admins). It encompasses registration for new Travellers, secure login, and session management. This feature is fundamental for personalized services, booking integrity, and administrative security.

Priority: High

3.1.2 Stimulus/Response Sequences

Sequence: User Login

1. **Stimulus:** The Traveller navigates to the login page and submits their credentials (email and password).
2. **Response:** The system validates the input format (email syntax, password length).
3. **Response:** The system hashes the provided password and compares it against the stored hash in the PostgreSQL database.
4. **Response:** If credentials match, the system generates a secure session token and redirects the Traveller to the dashboard or previously requested page.
5. **Response:** If credentials do not match, the system displays a generic "Invalid credentials" error message.

3.1.3 Functional Requirements

Requirement	Description
REQ-1	The system shall allow a new Traveller to register by providing a unique email address, a password (minimum 8 characters), and full name.
REQ-2	The system shall securely store all user passwords using the bcrypt hashing algorithm.
REQ-3	The system shall authenticate a user by verifying the submitted email and password against stored credentials.
REQ-4	Upon successful login, the system shall create a persistent, secure session token for the user.
REQ-5	The system shall provide a "Forgot Password" mechanism that sends a secure, time-limited link to the user's registered email address for password reset.
REQ-6	The system shall automatically terminate a user session after 30 minutes of inactivity.

3.2 Package Browsing

3.2.1 Description and Priority

The Package Browsing feature enables Travellers to discover, search, and view detailed information regarding available travel packages. This feature is crucial for converting casual visitors into engaged users and facilitating the core business function. It relies heavily on efficient database querying and a high-quality frontend presentation.

Priority: High

3.2.2 Stimulus/Response Sequences

Sequence: Advanced Package Search

1. **Stimulus:** The Traveller accesses the search page and enters search parameters (e.g., Destination: "Paris", Start Date: Date, Price Range: "$500-$1500").
2. **Response:** The Frontend transmits the search query to the Backend API.
3. **Response:** The Backend executes an optimized database query against the PostgreSQL database to retrieve all matching packages based on the criteria.
4. **Response:** The Backend processes the results, formats the data, and sends a JSON response back to the Frontend.
5. **Response:** The Frontend renders the results on a dedicated search results page, displaying package name, a thumbnail image, price, and a short description.

3.2.3 Functional Requirements

Requirement	Description
REQ-7	The system shall allow users to search for packages based on destination, travel dates, price range, and duration.
REQ-8	The system shall display detailed information for each package, including itinerary, inclusion/exclusion lists, pricing breakdown, and a gallery of high-resolution images.
REQ-9	The system shall provide filtering options for search results, allowing sorting by price (low-to-high, high-to-low), user rating, and

Requirement	Description
	departure date.
REQ-10	The system shall indicate real-time package availability status (e.g., "Available," "Limited Seats," or "Sold Out").
REQ-11	The system shall allow unauthenticated users to browse packages but shall prompt them to log in or register before proceeding to the booking stage.

3.3 Booking System

3.3.1 Description and Priority

The Booking System is the transactional core of TravelSphere, allowing authenticated Travellers to reserve a travel package and complete the financial transaction. It must ensure data integrity, accurately manage inventory, and securely handle sensitive payment information.

Priority: High

3.3.2 Stimulus/Response Sequences

Sequence: Package Reservation and Payment

1. **Stimulus:** An authenticated Traveller clicks the "Book Now" button on a package detail page.
2. **Response:** The system verifies the Traveller's session and the package's immediate availability in the database, initiating a temporary reservation hold (e.g., for 10 minutes).
3. **Stimulus:** The Traveller completes the multi-step form, providing necessary passenger details and selecting optional services.
4. **Response:** The system calculates the final, precise cost, including taxes and selected options, and presents it to the Traveller.
5. **Stimulus:** The Traveller enters payment details and clicks "Confirm and Pay."
6. **Response:** The system securely transmits payment data to the external Payment Gateway.
7. **Response:** Upon receiving a successful transaction confirmation from the gateway, the system updates the booking status to "Confirmed," decrements

the package's available inventory count in the database, and sends a confirmation email to the Traveller.

3.3.3 Functional Requirements

Requirement	Description
REQ-12	The system shall provide a multi-step checkout process for collecting passenger information and payment details.
REQ-13	The system shall integrate with a secure third-party payment gateway to accept major credit cards.
REQ-14	The system shall generate a unique booking ID for every confirmed transaction.
REQ-15	The system shall automatically send an email confirmation detailing the itinerary and booking status to the Traveller upon successful payment.
REQ-16	The system shall update the package's inventory count immediately upon confirmation of a successful booking.
REQ-17	The system shall store all non-sensitive booking details (e.g., total price, date, Traveller ID) in the PostgreSQL database.

3.4 Review System

3.4.1 Description and Priority

The Review System allows verified Travellers to provide feedback on their completed packages through ratings and written reviews. This content enhances package credibility and assists other Travellers in their decision-making process. The system must ensure that only actual customers can submit reviews.

Priority: Medium

3.4.2 Stimulus/Response Sequences

Sequence: Submitting a Review

1. **Stimulus:** A Traveller, whose booking for a specific package is marked as "Completed," navigates to the review submission page for that package.
2. **Response:** The system verifies the Traveller's identity and booking history to confirm eligibility for the review.
3. **Stimulus:** The Traveller submits a numerical rating (1-5 stars) and a textual review (minimum 50 characters, maximum 1000 characters).
4. **Response:** The system validates the content for length and basic profanity filters.
5. **Response:** The system saves the review and rating in the database, marking it as "Pending Approval." The system recalculates the package's average rating.
6. **Response:** The system displays a success message to the Traveller.

3.4.3 Functional Requirements

Requirement	Description
REQ-18	The system shall restrict review submission only to authenticated Travellers who have a "Completed" booking status for the specific package.
REQ-19	The system shall allow Travellers to submit a star rating on a scale of 1 to 5.
REQ-20	The system shall allow Travellers to submit a textual review with a maximum length of 1000 characters.
REQ-21	The system shall automatically calculate and display the average rating for each travel package based on all approved reviews.
REQ-22	The system shall initially mark all submitted reviews as "Pending Approval" for Admin moderation.

3.5 Admin Management

3.5.1 Description and Priority

The Admin Management feature provides a secure, dedicated dashboard for system administrators to perform critical maintenance, package content updates, and oversight tasks, ensuring the operational health and content quality of TravelSphere.

Priority: High

3.5.2 Stimulus/Response Sequences

Sequence: Package Creation

1. **Stimulus:** An authenticated Admin logs into the Admin Management dashboard and navigates to the "Create New Package" section.
2. **Response:** The system presents a form requiring mandatory details: Package Name, Destination, Price, Duration, Start Date, and available inventory count.
3. **Stimulus:** The Admin fills out the required information and uploads necessary media files (images).
4. **Response:** The system validates the input (e.g., positive price, future date) and creates a new entry in the PostgreSQL database for the package.
5. **Response:** The system displays a confirmation message with the new Package ID.

3.5.3 Functional Requirements

Requirement	Description
REQ-23	The system shall require a separate, high-security login for access to the Admin Management dashboard.
REQ-24	The Admin shall be able to perform CRUD operations (Create, Read, Update, Delete) on all travel packages.
REQ-25	The Admin shall have the capability to view a list of all current bookings, filtered by status (Confirmed, Pending, Cancelled).
REQ-26	The Admin shall be able to manually update the status of any booking record.
REQ-27	The Admin shall be able to approve or reject reviews marked as "Pending Approval" before they are visible to the public.
REQ-28	The Admin interface shall include tools for searching and viewing all registered Traveller accounts.

4. External Interface Requirements

4.1 User Interfaces

The TravelSphere system will feature two distinct, role-based User Interfaces (UIs): the primary Traveller UI and the restricted Admin Management Dashboard. The **Traveller UI** will utilize a clean, modern, single-page application (SPA) design implemented with React.js, emphasizing intuitive navigation and high responsiveness across devices. A consistent global navigation bar will provide immediate access to core functions: Search, Bookings, Profile, and Authentication (Login/Register). All forms (registration, login, booking steps) will employ client-side validation to enhance user experience by providing immediate feedback and minimizing server load, followed by robust server-side validation. The core user experience prioritizes simplicity in the package selection and booking workflow, minimizing the number of clicks required to complete a transaction. The **Admin Management Dashboard** will be a separate, secure interface characterized by data-heavy views (tables, reports) and form-centric workflows for content management (CRUD operations on packages) and operational oversight (booking status updates, review moderation). Its design will prioritize functional efficiency, data density, and security over aesthetic appeal, utilizing standardized table components for filtering, sorting, and pagination of large datasets.

4.2 Hardware Interfaces

As a web-based application, TravelSphere does not directly interface with proprietary hardware devices. Its hardware interface requirement is defined by its compatibility and responsiveness across standard computing and mobile devices. The frontend, built using responsive design principles, must render correctly and maintain full functionality on diverse screen sizes and resolutions, ranging from high-resolution desktop monitors and laptops to various form factors of tablets and mobile phones (iOS and Android). The system relies solely on the user's standard input peripherals (keyboard, mouse, or touch screen) and output display. Performance testing will ensure that load times and interaction responsiveness remain acceptable across a range of common user hardware specifications, preventing slow performance from being attributed to device incompatibility.

4.3 Software Interfaces

TravelSphere's software interfaces are primarily internal and inter-system. **Internal Software Interface:** The Frontend (React.js) communicates exclusively with the Backend (Node.js/Express) via a comprehensive set of **RESTful APIs**. This interface dictates the structure of all data exchange, utilizing JSON payloads for requests and responses. **Database Interface:** The Backend layer interfaces directly with the PostgreSQL relational database management system using secure connection pooling and an Object-Relational Mapper (ORM) to manage data persistence, ensuring that all data manipulation adheres to the defined database schema. **External Software**

Interface: The most critical external interface is the connection to a **third-party Payment Gateway API**. This connection will involve the secure transmission of transaction details (amount, currency, tokenized card information) and the receipt of transaction status (Success, Failure, Pending). This interface must strictly adhere to industry standards (e.g., PCI DSS compliance guidelines), necessitating secure data tokenization and encrypted communication, as sensitive payment data will not be stored directly on TravelSphere servers.

4.4 Communication Interfaces

All system communication, both internal and external, relies fundamentally on standard internet protocols. **Protocol:** The system will exclusively use **HTTPS/SSL** for all data transfer between the client and the server to guarantee confidentiality and integrity against eavesdropping and man-in-the-middle attacks. **Architecture:** All communication between the Presentation Tier and the Application Tier is conducted via **RESTful Web Services**. These APIs use standard HTTP methods (GET, POST, PUT, DELETE) for corresponding CRUD operations, ensuring a stateless, client-server architecture. **Data Exchange:** All transactional and data exchange payloads are formatted using the **JSON (JavaScript Object Notation)** standard due to its lightweight nature and native compatibility with the JavaScript-based technology stack. Communication with the external Payment Gateway will also be over HTTPS, conforming to the gateway's specific protocol requirements (often JSON or XML).

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The TravelSphere system must demonstrate high performance to ensure a satisfying user experience. **Response Time:** A critical performance constraint dictates that all primary pages, specifically the search results page and individual package detail pages, must load and be fully interactive within a maximum of **three (3) seconds** under normal operating conditions. Furthermore, the submission and processing of the final booking transaction, including secure communication with the payment gateway, must be completed and confirmed within **ten (10) seconds**. **System Speed and Efficiency:** The backend database queries for package searching and filtering must execute in under **500 milliseconds**, optimized through appropriate indexing and query planning. The system must support a concurrent load of at least **500 authenticated users** without experiencing performance degradation or exceeding the specified response time limits.

5.2 Safety Requirements

Safety requirements focus on protecting the system from internal and external failures that could lead to data loss or user dissatisfaction. **Error Handling:** The system must implement robust, centralized error handling in the backend to gracefully manage

application exceptions, database connection failures, and API timeouts. User-facing error messages must be clear, non-technical, and provide actionable steps (e.g., "Please try again later," or "Payment failed: check your card details"). **Data Protection:** Mechanisms must be in place to prevent accidental or malicious data modification. For example, once a booking is confirmed, critical details (price, package ID) should be immutable except through a restricted administrative process. **System Failure Prevention:** The system architecture must incorporate redundancy measures, such as database replication and load balancing, to prevent a single component failure from causing complete system downtime. A structured rollback plan must be established for database transactions, especially during the booking process, to ensure atomicity (ACID properties) and prevent inconsistent data states.

5.3 Security Requirements

Security is paramount given the handling of personal and financial information. **Authentication:** User authentication (Traveller and Admin) must enforce strong password policies (minimum 8 characters, alphanumeric and special characters) and utilize the **bcrypt** hashing algorithm for irreversible, secure password storage. Failed login attempts must be throttled to mitigate brute-force attacks. **Authorization:** Access control must be strictly enforced, based on the user's role (Traveller or Admin). The Admin Management Dashboard must be inaccessible to regular Travellers, and API endpoints must validate the user's session token and permissions before executing any privileged operation (e.g., package creation, booking status update). **Data Privacy:** All personally identifiable information (PII) must be stored securely in the PostgreSQL database with appropriate encryption for sensitive fields where required. The system must never store raw credit card numbers; instead, it must rely on the Payment Gateway's tokenization service for all transactions, ensuring PCI DSS compliance.

5.4 Software Quality Attributes

The following quality attributes define the long-term viability and usability of TravelSphere. **Reliability:** The system must achieve an uptime of **99.9%** per month, excluding scheduled maintenance periods. Data integrity must be ensured through transactional database logic. **Usability:** The Traveller UI must be accessible and usable by individuals with basic computer literacy, achieving a high score on standard usability metrics (e.g., task completion rate, time on task). The booking process should require a maximum of five distinct steps from package selection to confirmation. **Scalability:** The three-tier architecture must support horizontal scaling. The database must be structured to handle potential growth in users and bookings by optimizing indexing and partitioning strategies. The backend must be deployable across multiple server instances via load balancing. **Maintainability and Flexibility:** The code base must be well-documented, modularized, and follow standardized coding practices to allow new features to be integrated and existing modules to be updated with minimal impact on other components. Flexibility is demonstrated by the system's ability to

easily accommodate integration with new third-party travel APIs or alternative payment gateways in future expansion phases.

6. Other Requirements

6.1 Database Requirements

The PostgreSQL database must adhere to strict structural and integrity requirements. The schema must be normalized to at least **Third Normal Form (3NF)** to eliminate transitive dependencies and minimize data redundancy. Referential integrity must be strictly enforced using foreign key constraints between tables (e.g., Booking records must reference a valid Traveller ID and a valid Package ID). All mission-critical data fields must enforce data type and length constraints to maintain consistency. Database indexing will be optimized for frequently searched fields such as package destination, date, and user email address to meet performance requirements.

6.2 Backup and Recovery

A comprehensive backup and recovery strategy must be implemented to ensure business continuity. **Backup:** Full, incremental backups of the entire PostgreSQL database must be performed daily and stored securely on an off-site, redundant storage solution. Transaction logs must be backed up hourly to minimize potential data loss. **Recovery:** The system must be capable of recovering from a catastrophic failure (e.g., server crash, major data corruption) to the point of the last hourly transaction backup within a Recovery Time Objective (RTO) of **four (4) hours** and a Recovery Point Objective (RPO) of **one (1) hour** of data loss. The recovery plan must be documented, tested, and regularly audited.

6.3 Future Scalability and Expansion Support

The current design, utilizing React.js, Node.js/Express, and PostgreSQL, is specifically chosen for its inherent scalability and modern development practices. Future expansion support is anticipated in several areas: **API Design:** The RESTful API architecture facilitates easy integration of new frontends (e.g., native mobile applications) without altering the core business logic. **Modular Backend:** The Express framework promotes the creation of modular, microservice-like components for handling distinct features (e.g., a dedicated service for notifications, a separate service for analytics), enabling independent scaling of high-demand features. **Internationalization:** The current database schema and frontend design should be planned to accommodate future requirements for multi-language support (i18n) and multi-currency processing with minimal schema restructuring.

Appendix A: Glossary

Term	Definition
Traveller	An end-user of the TravelSphere system who registers, logs in, browses packages, and initiates bookings.
Admin	A system administrator with high-level access to the dedicated management dashboard for content and operational control.
Booking	A confirmed and paid-for reservation of a travel package, represented by a unique Booking ID and a specific status (e.g., Confirmed, Cancelled).
Package	A single, combinational travel offering defined by the Admin, typically including itinerary, dates, price, and available inventory (e.g., "7-Day Paris Explorer").
Review	User-generated feedback, consisting of a numerical rating (1-5 stars) and a textual comment, submitted by a verified Traveller after trip completion.
Authentication	The process by which the system verifies a user's identity, typically via secure login credentials (email and password).

Appendix B: Analysis Models

Use Case Diagram

A high-level Use Case Diagram illustrates the system's functional scope by modeling the interactions between the two primary actors (Traveller and Admin) and the system.

- **Traveller Use Cases:** Browse Packages, Search/Filter Packages, Register Account, Login, Manage Profile, Initiate Booking, Submit Payment, View Booking History, Submit Review, Reset Password.
- **Admin Use Cases:** Secure Admin Login, Manage Packages (CRUD), View All Bookings, Update Booking Status, Moderate Reviews, Manage User Accounts.

Entity-Relationship (ER) Diagram

The ER Diagram defines the logical structure of the PostgreSQL database, illustrating the relationship between key entities. Key entities include:

- **Traveller (One-to-Many with Booking):** Attributes include TravellerID, Name, Email (Unique), PasswordHash.
- **Package (One-to-Many with Booking, One-to-Many with Review):** Attributes include PackageID, Name, Destination, Price, InventoryCount, Description.
- **Booking (Many-to-One with Traveller, Many-to-One with Package):** Attributes include BookingID (Primary Key), BookingDate, TotalPrice, Status, TravellerID (Foreign Key), PackageID (Foreign Key).
- **Review (Many-to-One with Traveller, Many-to-One with Package):** Attributes include ReviewID, Rating, Text, Status (Pending/Approved), TravellerID (Foreign Key), PackageID (Foreign Key).

System Architecture Diagram

The system follows a standard **Three-Tier Architecture**:

1. **Presentation Tier (Client/Frontend):** React.js application running on the user's web browser, responsible for rendering the UI and handling user interaction. Communicates via HTTPS/REST.
2. **Application Tier (Backend/Business Logic):** Node.js with Express framework, hosts the RESTful APIs. It handles authentication, authorization, business logic (e.g., price calculation, inventory check), and acts as the intermediary between the Frontend and the Database.
3. **Data Tier (Database):** PostgreSQL database server, responsible for persistent storage of all system data. Accessed securely only by the Application Tier.

Appendix C: Issues List

The following issues represent outstanding items, potential risks, or features deferred to a later phase of development:

Issue/Risk ID	Description	Priority
IS-1	**Detailed Payment System Integration:** The current SRS confirms reliance on a third-party gateway, but the final choice of provider (e.g., Stripe, PayPal) and detailed API implementation specifications remain pending selection and	High

Issue/Risk ID	Description	Priority
	integration.	
IS-2	**User Notification System:** The feature to notify Travellers of booking status changes, promotional offers, or system updates (beyond the core booking confirmation email) is not yet fully specified or implemented.	Medium
IS-3	**Future Mobile Application Support:** The current system is a responsive web application. Support for dedicated native iOS and Android applications is a planned future expansion (Phase 2 or 3) and is not covered by the current API design requirements.	Low
IS-4	**Advanced Analytics Dashboard:** Detailed reporting and business intelligence (BI) tools for the Admin (e.g., revenue tracking, conversion rates) are not currently required but will be necessary for scaling the business.	Low